

Jaguar Robot Control System

Web-based robot control application for MTSU Jaguar Robot with real-time navigation, camera feed, sensor data fusion, and autonomous features.

 **Returning users:** Jump to [Already Set Up? Quick Start!](#) below

Project Structure

```
├── PI/                # Raspberry Pi sensor collection
|
|   ├── harry_SensorImproved.py # IMU/GPS data publisher (5Hz MQTT)
|   └── [sensor scripts]      # BNO055 IMU + GPS integration
|
|   ├── appVersions/         # Flask application evolution
|   │   ├── app.py           # Base version - manual control
|   │   ├── appUpdated.py    # + Enhanced sensor fusion
|   │   ├── appAutoNav.py    # + Autonomous navigation
|   │   └── appAutoCapture.py # + Auto photo capture (latest)
|
|   ├── static/              # Web assets
|   │   ├── robot.png        # Robot icon for map visualization
|   │   └── [images]
|
|   ├── templates/           # HTML interfaces
|   │   ├── index.html       # Main map & control dashboard
|   │   └── camera.html      # Full-screen camera view
|
|   ├── setup/               # Installation & config scripts
|   │   └── setup.sh          # Automated dependency installer
```

```
| └─ network_config.py    # Robot network switcher
|
| └─ start.sh            # Quick launch script
└─ README.md            # This file
```

Quick Setup

First-Time Setup (New Machine)

For Development/Testing (download robot codebase):

Download Jaguar files from Google Drive, then:

```
cd /path/to/MTSUJaguarRobot
```

```
./setup/setup.sh start    # One command - sets up everything and starts app
```

For Robot Control (first time using robot hardware):

1. Setup dependencies while on internet

```
cd /path/to/MTSUJaguarRobot
```

```
./setup/setup.sh          # Downloads all dependencies
```

2. Switch to robot WiFi

Connect to: DirJaguar / drrobotdrrobot

3. Configure robot network

```
python3 setup/network_config.py robot
```

4. Start controlling robot

```
./start.sh
```

Already Set Up? Quick Start!

If this machine already has the robot code installed:

1. **Open terminal** in project folder:
2. `cd /path/to/MTSUJaguarRobot`
3. **Connect to robot WiFi** (if controlling robot):
 - WiFi: DirJaguar / Password: drrobotdrrobot
4. **Start the app** (one command):
5. `./start.sh`
6. **Control the robot:**
 - **Ctrl+click** the link: <http://127.0.0.1:5014>
 - Use WASD keys to control
7. **Auto-Capture photos:**
 - Press **Camera** button
 - Press **Photo** button (starts 3-second auto-capture)
 - Press **Photo** again to stop

Alternative start:

```
python3 src/app.py    # Direct start without launcher
```

Then open: **<http://127.0.0.1:5014>**

Controls & Features

Keyboard Controls

Key	Action
------------	---------------

W	Forward
----------	---------

S	Backward
----------	----------

A	Turn Left
----------	-----------

D	Turn Right
----------	------------

Space Emergency Stop

Key Action

N Toggle Auto Navigation

Web Interface Features

- **Real-time Control** - WASD keyboard/button controls
- **Live Camera Feed** - MJPEG stream from robot camera
- **Interactive Map** - Leaflet.js with GPS/IMU tracking
- **Waypoint Navigation** - Click-to-set autonomous following
- **Auto Navigation** - Back-and-forth patrol mode
- **Auto Photo Capture** - Time-lapse photography (3s intervals)
- **Sensor Fusion** - GPS + IMU data integration
- **PID Tuning** - Real-time parameter adjustment
- **Safety Features** - Auto-stop, rate limiting, command persistence

Button Functions

- **Connect Robot** - Establish socket connection (green = connected)
- **Camera** - Toggle live camera view
- **Photo** - Start/stop auto-capture mode
- **Point** - Enable waypoint placement on map
- **Go/Stop** - Control autonomous waypoint following
- **PID** - Open tuning panel (Kp, Ki, Kd, Speed 0-500)
- **Data** - View raw MQTT sensor stream
- **Lock Heading** - Map rotates with robot orientation
- **Navigation** - Toggle back-and-forth autonomous mode

Robot Network Setup

 **Important: Setup dependencies on internet first, then switch to robot WiFi**

Complete Robot Setup Workflow

1. **Setup on Internet** (download dependencies):
2. `./setup/setup.sh` # Must be done on internet connection
3. **Switch to Robot WiFi:**
 - Connect to: DirJaguar / drrobotdrrobot
4. **Configure Robot Network:**
5. `python3 setup/network_config.py robot`
6. **Start Robot Control:**
7. `./start.sh`
8. **Switch Back to Internet** (when done):
9. `python3 setup/network_config.py internet`

Manual Network Configuration

If automated script doesn't work:

macOS/Linux:

Set static IP

```
sudo ifconfig en0 192.168.0.109 netmask 255.255.255.0
```

```
sudo route add default 192.168.0.1
```

Set DNS

```
echo "nameserver 192.168.0.1" | sudo tee /etc/resolv.conf
```

Windows:

Network Settings > Change adapter options

Right-click WiFi > Properties > IPv4 > Use the following IP:

IP: 192.168.0.109

Subnet: 255.255.255.0

Gateway: 192.168.0.1

DNS: 192.168.0.1

Required Network Settings:

- IP Address: 192.168.0.109
- Subnet Mask: 255.255.255.0
- Gateway: 192.168.0.1
- DNS Server: 192.168.0.1

Robot Connection Details:

- Robot Control: 192.168.0.60:10001
 - Camera Feed: 192.168.0.65:8081
 - MQTT Broker: 192.168.1.103:1883
-

System Components

Raspberry Pi Sensor Module (PI/)

Purpose: Real-time sensor data collection and MQTT publishing

Hardware:

- **BNO055 IMU** (9-DOF)
 - Heading, roll, pitch (Euler angles)
 - Linear acceleration (gravity compensated)
 - I2C communication
- **GPS Module** (NMEA compatible)
 - Position: latitude, longitude, altitude
 - Fix quality and satellite count
 - Serial communication (9600 baud)

Features:

- 5Hz publishing rate
- Automatic sensor initialization

- Multi-port GPS detection
- Graceful degradation (works with missing sensors)

MQTT Topics:

- IMU/data - Combined IMU + GPS payload
- GPS/data - GPS-only data stream

Dependencies:

`pip3 install adafruit-circuitpython-bno055 paho-mqtt pyserial`

Configuration:

`MQTT_BROKER = "192.168.1.103" # Update with your broker`

`GPS_PORT = "/dev/ttyUSB0" # Auto-detected: USB0, AMA0, serial0`

Run:

`cd PI/`

`python3 harry_SensorImproved.py`

Flask Application Versions (appVersions/)

All versions share core features:

- Socket-based robot control
- MQTT data integration
- Real-time map visualization
- WASD keyboard controls
- PID parameter tuning

app.py - Base Version

Foundational manual control and data display

appUpdated.py - Enhanced Sensor Fusion

- **Weighted Fusion:** GPS (reliable position) + IMU (fast heading)
- **Dynamic Offset Correction:** Learns heading error over 5-100m

- **Static Detection:** Zeros acceleration when not moving
- **Friction Compensation:** Increases speed if turning but not rotating
- **Position Interpolation:** Smooth updates between GPS fixes

appAutoNav.py - Autonomous Navigation

Previous features +

- **Forward/Backward Cycling:** Configurable duration (default 20s)
- **Command Persistence:** 500ms refresh rate
- **Movement Verification:** GPS-based distance monitoring
- **Enhanced Stopping:** Multiple stop commands for reliability
- **Speed Ramping:** Smooth acceleration/deceleration

Usage:

- Press **N** or click **Navigation** button
- Robot alternates: Forward 20s → Pause 1s → Backward 20s

appAutoCapture.py - Auto Photo Capture 🌟 (Latest)

Previous features +

- **Automatic Photography:** 3-second capture intervals
- **Local Storage:** Timestamped filenames (photo_YYYYMMDD_HHMMSS.jpg)
- **Photo Management:** List and delete via API
- **Status Indicators:** Visual feedback during capture
- **Storage Directory:** captured_photos/

Photo Capture:

1. Click **Camera** to start video feed
2. Click **Photo** to begin auto-capture (button turns yellow)
3. Photos saved every 3 seconds
4. Click **Photo** again to stop (button returns to blue)

API Endpoints:

- POST /capture-photo - Single capture
 - GET /photos - List all saved photos
 - DELETE /delete-photo/<filename> - Remove photo
-

Web Interface (templates/)

index.html - Main Control Dashboard

Interactive Map (Leaflet.js):

- Real-time robot position tracking
- GPS accuracy circle (pulses on update)
- Robot icon with heading rotation
- Offline grid layer (no internet needed)
- Heading lock mode (map rotates with robot)
- Distance measurement tools

Waypoint Navigation System:

- Click map to set waypoints
- Visual path with distance labels
- 1-meter route interpolation
- PID-based autonomous following
- Arrival detection (1m radius)

Control Panels:

- **PID Tuning:** Kp, Ki, Kd adjustment
- **Speed Control:** 0-500 range
- **Navigation Duration:** Cycle time config
- **Data Visualization:** Real-time MQTT stream
- **Camera Integration:** Live MJPEG feed

Compass Display:

- 360° visual heading indicator
- Red arrow rotates with IMU data
- Compass rose background

Advanced Features:

- Route interpolation (1m checkpoints)
- Outside-route detection and correction
- Multi-segment path planning
- Ghost line preview while placing waypoints

camera.html - Camera View

Full-screen camera feed with return navigation

Technical Details

Sensor Fusion Algorithm

Weighted Fusion Approach:

Position fusion (40% IMU, 60% GPS)

$\text{fused_lat} = 0.4 * \text{imu_lat} + 0.6 * \text{gps_lat}$

$\text{fused_lon} = 0.4 * \text{imu_lon} + 0.6 * \text{gps_lon}$

Heading fusion (adjusts based on movement)

if moving_fast:

$\text{fused_heading} = 0.4 * \text{imu_heading} + 0.6 * \text{gps_bearing}$

else:

$\text{fused_heading} = 0.6 * \text{imu_heading} + 0.4 * \text{gps_bearing}$

Dynamic Heading Correction:

1. Monitor GPS bearing over distance
2. Detect consistent direction (< 10° variation)

3. Calculate IMU vs GPS heading difference
4. Apply trust ratio based on distance traveled:
 - 5m traveled → begin correction
 - 100m traveled → 100% trust GPS bearing
5. Update IMU heading offset: $\text{imu_heading} += \text{learned_offset}$

Static Detection:

- GPS movement < 2m → stationary
- Acceleration magnitude < 100 → stationary
- When static: zero out acceleration values
- Prevents IMU drift accumulation

Friction Logic:

- If turning commands sent but heading unchanged for 5+ cycles
- Increase speed by +50 to overcome friction
- Resets when rotation detected

Navigation System

Enhanced Movement Control:

- **Command Persistence:** Resend every 500ms
- **Multiple Stop Commands:** 3× stop for reliability
- **Speed Ramping:** 1-second ramp up/down
- **Movement Verification:** GPS distance monitoring

Waypoint Following (PID):

$\text{error} = \text{target_bearing} - \text{current_heading}$

$\text{integral} += \text{error}$

$\text{derivative} = \text{error} - \text{previous_error}$

$\text{output} = K_p * \text{error} + K_i * \text{integral} + K_d * \text{derivative}$

```
if abs(error) < 10°:
```

```
    move_forward()
```

```
elif output > 0:
```

```
    turn_right()
```

```
else:
```

```
    turn_left()
```

Route Interpolation:

- Interpolate 1-meter checkpoints between waypoints
- Detect if robot is outside 1m radius of route
- Correct back to nearest checkpoint
- Prevents shortcuts and maintains path

Auto-Capture System

Implementation:

```
// Start auto-capture
```

```
captureInterval = setInterval(capturePhoto, 3000);
```

```
// Capture single frame
```

```
fetch('/capture-photo', {method: 'POST'})
```

```
    .then(response => response.json())
```

```
    .then(data => console.log('Saved:', data.filename));
```

Server-side (Flask):

```
# Capture from latest camera frame
```

```
timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
```

```
filename = f'photo_{timestamp}.jpg'
```

```
cv2.imwrite(filepath, latest_frame)
```

Features:

- Non-blocking capture (doesn't interrupt control)
- Automatic filename generation
- File size reporting
- Error handling for missing frames

API Reference

Robot Control

Endpoint	Method	Description
/connect-robot	POST	Establish socket connection
/robot-cmd	POST	Send movement command (w/a/s/d/stop/n)
/robot-autopilot	POST	Autopilot actions (forward/rotateLeft/rotateRight/stop)

Example:

```
curl -X POST http://localhost:5014/robot-cmd \  
-H "Content-Type: application/json" \  
-d '{"cmd": "w"}'
```

Data & Configuration

Endpoint	Method	Description
/data	GET	Current fused sensor data
/mqtt-data	GET	Recent MQTT message buffer (40 msgs)
/pid-update	POST	Update PID parameters
/navigation-config	GET/POST	Navigation settings
/navigation-status	GET	Current navigation state

PID Update Example:

```
curl -X POST http://localhost:5014/pid-update \  

```

```
-H "Content-Type: application/json" \
```

```
-d '{"kp": 1.5, "ki": 0.1, "kd": 0.5, "speed": 200, "nav": 25}'
```

Camera (appAutoCapture.py)

Endpoint	Method	Description
/toggle-camera	POST	Start/stop camera stream
/camera-stream	GET	MJPEG video stream
/capture-photo	POST	Capture single photo
/photos	GET	List all saved photos
/delete-photo/<filename>	DELETE	Remove specific photo

Photo Capture Example:

Capture photo

```
curl -X POST http://localhost:5014/capture-photo
```

List photos

```
curl http://localhost:5014/photos
```

Delete photo

```
curl -X DELETE http://localhost:5014/delete-photo/photo_20250107_143022.jpg
```

Development Mode

On regular internet connection, the app works for:

- Testing web interface
- Developing new features
- UI demonstrations
- Learning the codebase

MQTT timeouts are normal without robot connection.

To control actual robot:

1. Follow "Robot Network Setup" above
 2. Click **Connect Robot** (green = connected, red = failed)
 3. Click **Camera** for live feed
 4. Click **PID** and set speed (0-500 range)
-

Troubleshooting

Common Issues

GPS Not Locking:

- Ensure clear sky view (outdoors)
- Wait 30-60 seconds for initial fix
- Check serial port: `ls /dev/tty*`
- Try multiple ports: USB0, AMA0, serial0

IMU Calibration:

- Move robot in figure-8 pattern
- Calibration status shown in console
- May take 30 seconds after startup
- Check I2C connection: `i2cdetect -y 1`

MQTT Connection Failed:

- Verify broker IP: 192.168.1.103
- Check firewall: `sudo ufw status`
- Test connection: `mosquitto_sub -h 192.168.1.103 -t "IMU/data"`
- Ensure mosquitto running: `sudo systemctl status mosquitto`

Robot Not Responding:

- Verify IP: 192.168.0.60:10001

- Test connection: telnet 192.168.0.60 10001
- Check console for socket timeout errors
- Click **Connect Robot** button again

Camera Stream Issues:

- Verify camera URL: http://192.168.0.65:8081
- Test in browser first
- Check credentials: root:drrobot
- Ensure sufficient bandwidth

Port Already in Use:

Find process using port 5014

```
lsof -i :5014
```

Kill process

```
kill -9 <PID>
```

Or use different port

```
python3 src/app.py --port 5015
```

Python Issues:

Check version (need 3.10+)

```
python3 --version
```

Reinstall dependencies

```
pip3 install --force-reinstall -r requirements.txt
```

Test setup

```
python3 setup/test_setup.py
```


Network Configuration Problems:

Reset network to defaults

python3 setup/network_config.py internet

Flush DNS cache (macOS)

sudo dscacheutil -flushcache

Verify IP assignment

ifconfig en0 # macOS/Linux

ipconfig # Windows

⚠ Safety Notes

Always follow these guidelines:

- ✅ Monitor robot during autonomous operation
- ✅ Keep emergency stop (Space) ready
- ✅ Test in open area first
- ✅ Start with low speed (100-150)
- ✅ Verify GPS lock before waypoint navigation
- ⚠ Laser power set to 2W by default (safety)
- ⚠ Adjust speeds cautiously when tuning PID
- ⚠ Autonomous mode can be unpredictable
- ⚠ Always have clear line of sight to robot

Emergency Procedures:

1. Press **Space** for immediate stop
2. Press **Stop** button on interface

3. Click **Stop** in autopilot mode
4. Power off robot if unresponsive